Spring Boot 3.X 개발 기본

inky4832@daum.net

4. 데이터베이스 연동 (MyBatis)

- 데이터베이스 연동 기초
- MyBatis 프레임워크 연동

4.1 데이터베이스 연동 기초

데이터베이스

- 개념
  - 데이터를 체계적으로 저장하고 관리하는 시스템 (DBMS)
- 종류
  - 관계형 DB (RDBMS)
  - 비관계형 DB (NoSQL)

4.1 데이터베이스 연동 기초

관계형 데이터베이스 (RDBMS)

- 개념
  - 데이터를 테이블(table)형태로 저장 관리하는 데이터베이스 시스템
  - 테이블 간 관계(Relation)를 이용해서 데이터를 연결하고 중복을 최소화함
    ex. MySQL, MariaDB, PostgreSQL, Oracle, SQL Server

- 특징

구분	설명
데이터 구조 / 스키마	테이블(행/열) 기반 구조 / 고정된 스키마(구조) 필요
관계(모델)	외래키(Foreign Key)로 여러 테이블을 연결
쿼리 언어	SQL(Structured Query Language) 사용
트랜잭션 지원	ACID 원칙 (Atomicity, Consistency, isolation, Durability) 보장
예시	select * from users where id=1;

4.1 데이터베이스 연동 기초

관계형 데이터베이스 (RDBMS)

- 장점
  - 데이터 일관성(Consistency)과 무결성(Integrity) 보장
  - 복잡한 쿼리 작업이 가능
  - 트랜잭션 처리에 강함 ex. 은행, 결제시스템 등

- 단점
  - 스키마 변경이 어려움 (유연성이 낮음)
  - 대용량 데이터 및 분산 처리 상대적으로 어려움
  - 서버 분산 확장이 어렵고 비용이 큼

4.1 데이터베이스 연동 기초

비관계형 데이터베이스 (NoSQL: Not Only SQL)

- 개념
  - 전통적인 테이블 구조가 아닌 비정형/반정형 데이터 저장용 DBMS
  - Not Only SQL은 SQL뿐만 아니라 다양한 방식으로 데이터 저장/관리가 가능 의미
    ex. MongoDB, Redis, Cassandra, DynamoDB, Elasticsearch

- 특징

구분	설명
데이터 구조 / 스키마	문서(Document), 키-값(Key-value), 열(Column Family), 그래프 / 동적 스키마
관계(모델)	관계형 구조 없음 (JOIN 불가 또는 제한적)
쿼리 언어	SQL이 아닌 자체 API
트랜잭션 지원	BASE 원칙 (Basically Available, Soft state, Eventually consistent)
예시	{ "name": "홍길동", "age": 25, "skills": ["Java", "Spring"] }

4.1 데이터베이스 연동 기초

비관계형 데이터베이스 (NoSQL: Not Only SQL)

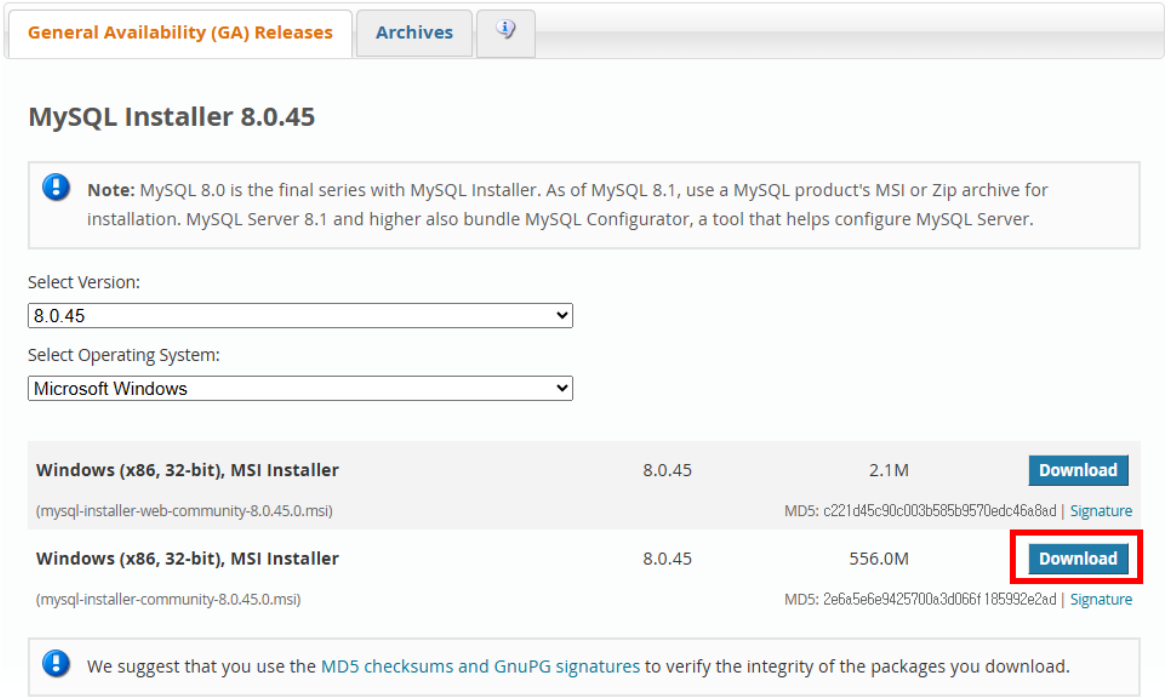
- 장점
  - 유연한 데이터 구조 ex. JSON, Key-value 등
  - 수평 확장성 뛰어남 (서버 추가만으로 확장 가능)
  - 대용량 트래픽 처리에 적합 ex, SNS, IoT, 로그 분석 등

- 단점
  - 데이터 일관성 보장 어려움
  - 복잡한 관계형 질의(JOIN) 불가
  - 각 DB 마다 쿼리 방식이 다름 (비표준화 방식)

4.1 데이터베이스 연동 기초

MySQL8 설치

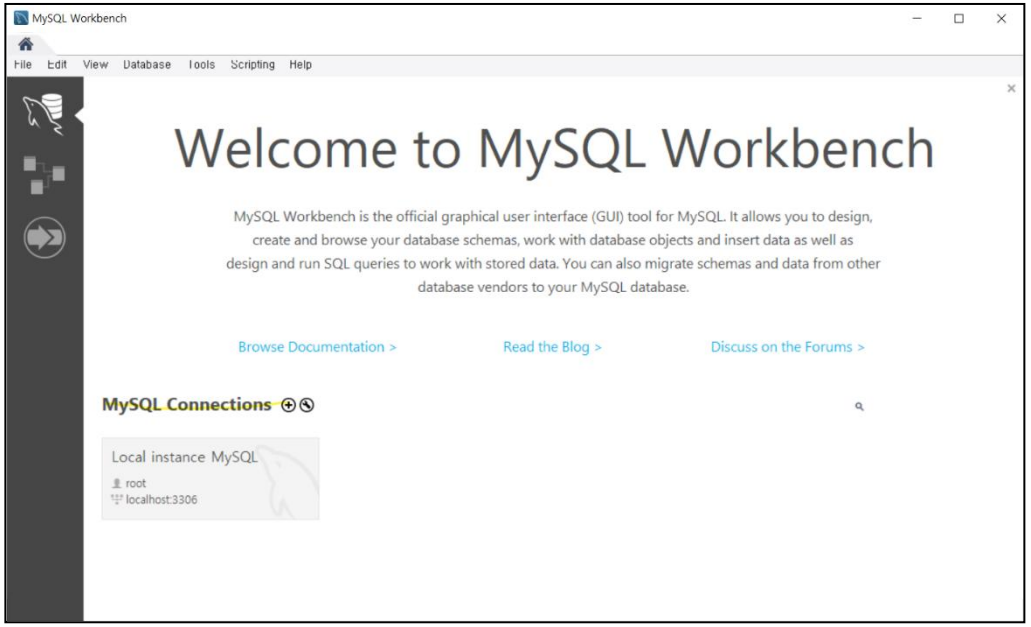
- 공식 사이트
  https://dev.mysql.com/downloads/installer/



3.1 데이터베이스 연동 기초

MySQL8 접속

- Workbench 툴 이용
  MySQL8 설치 시 자동으로 추가 설치됨



4.1 데이터베이스 연동 기초

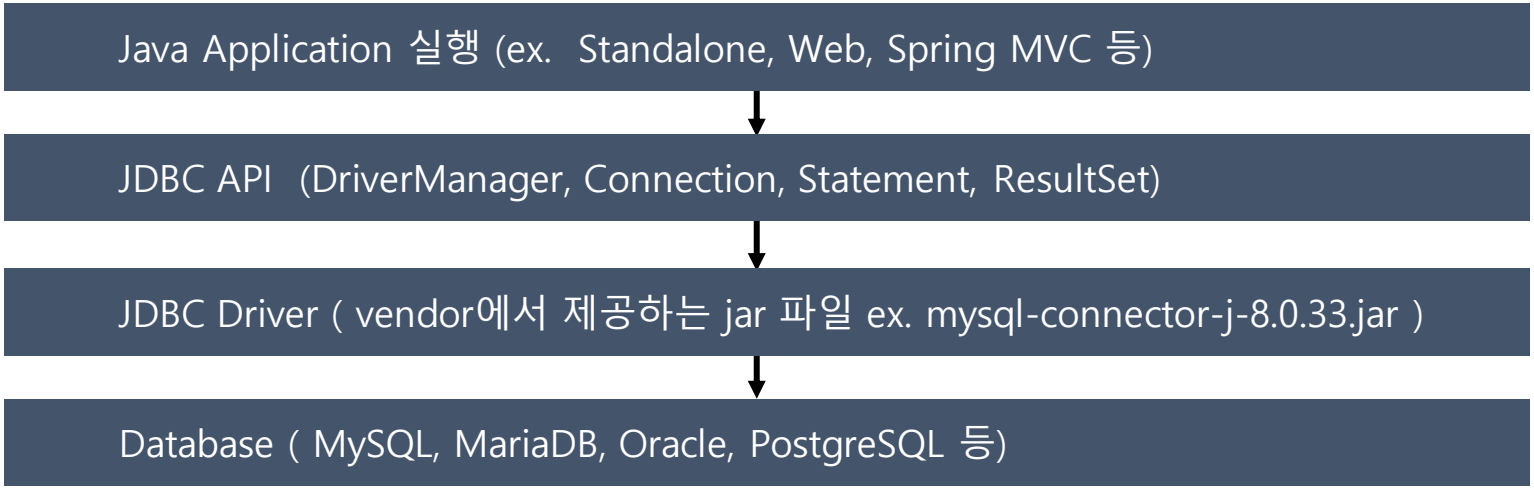
JDBC

- 개념
  - Java 어플리케이션에서 데이터베이스에 접근하고 SQL을 실행하기 위한 표준 인터페이스
  - DBMS 종류(MariaDB, Oracle 등)에 상관없이 일관된 코드로 DB 작업 수행 가능

- 특징
  - Java와 DB 간 연결 계층을 추상화 하여 개발자가 특정 DBMS에 종속되지 않도록 처리
  - JDBC는 모든 데이터베이스 연동 기술
    (MyBatis, Hibernate, Spring Data JPA)의 기본이 되는 기술 계층
  - **MyBatis와 JPA도 내부적으로 JDBC를 통해서 SQL 실행 및 커넥션 관리를 수행**

4.1 데이터베이스 연동 기초

JDBC 동작 흐름



4.1 데이터베이스 연동 기초

JDBC 동작 단계

단계	작업	설명
1	Driver 로드	Class.forName("com.mysql.cj.jdbc.Driver") 로 DB 드라이버 클래스 로딩
2	Connection 생성	DriverManager.getConnection(url, user, passwd) 로 DB 연결 생성
3	PreparedStatement	SQL 실행을 위한 객체 생성
4	SQL 실행 및 ResultSet 반환	SELECT: executeQuery() DML: executeUpdate()
5	결과처리(ResultSet)	결과집합을 반복문으로 순회하며 데이터 추출
6	자원 해제	close() 이용하여 ResultSet, PreparedStatement, Connection 순서로 처리

4.1 데이터베이스 연동 기초

JDBC 주요 클래스 및 인터페이스

클래스/인터페이스	설명	주요 메서드
DriverManger	벤더에서 제공해준 JDBC 드라이버(jar)를 관리하고 Connection 생성	getConnection()
Connection	데이터베이스 연결 세션 관리	prepareStatement()
PreparedStatement	정적 SQL 실행용 객체	executeQuery(), executeUpdate() setString(), setInt()
ResultSet	SQL 쿼리 결과를 행 단위로 저장하는 객체	next(), getString(), getInt()

4.1 데이터베이스 연동 기초

JDBC 코드 예시 (기본)

```
import java.sql.*;

public class JdbcExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String pass = "1234";

        try (Connection conn = DriverManager.getConnection(url, user, pass);
             PreparedStatement ps = conn.prepareStatement("SELECT * FROM users");
             ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                System.out.println(rs.getInt("id") + " : " + rs.getString("name"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

4.1 데이터베이스 연동 기초

JDBC 장단점

- 장점
  - 모든 DBMS에 대한 표준 API 제공
  - 직접 SQL 제어 가능
  - 의존성 최소화

- 단점
  - 반복적인 자원관리 코드
  - SQL문 하드코딩 (유지보수 어려움)
  - 객체 매핑 수동 처리
  - 예외처리 필수

4.1 데이터베이스 연동 기초

Spring JDBC

- 개념
  - 순수 JDBC의 보일러플레이트 코드를 제거해서 SQL에만 집중하도록 돕는 Spring의 고수준 유틸리티 (JdbcTemplate API 핵심)

- 특징

구분	설명
JDBC 코드 단순화	기존 Connection,Statement,ResultSet 생성과 해제 코드를 자동화
예외 처리 불필요	기존 SQLException을 Spring의 DataAccessException 계층으로 변환 (try~catch 불필요 및 DB 벤더 독립적인 예외 처리 가능)
손쉬운 트랜잭션 처리	선언적 트랜잭션(@Transactional) 이용한 commit/rollback 자동 처리
커넥션 풀 통합	HikariCP, DBCP2 등과 연동되어 Connection 재사용 및 성능 향상

4.1 데이터베이스 연동 기초

커넥션 (Connection)

- 개념
  - Java 어플리케이션이 DBMS와 통신하기 위해 맺는 네트워크 세션(연결)
  - SQL 전송, 결과 수신, 트랜잭션 수행 등 DB와의 모든 I/O 수행하는 통로

- 단점
  - 생성 비용이 매우 높음 (DB인증, 세션초기화 등 포함해서 수 ms ~ 수백 ms 소요)
  - 요청마다 Connection을 새로 생성하고 닫으면 성능 저하 및 리소스 낭비 발생
    ex. 1000명의 동시 접속자가 있다면, 1000개의 Connection 생성/종료 필요 (DB 부하 심각)

4.1 데이터베이스 연동 기초

커넥션 풀(Connection Pool)

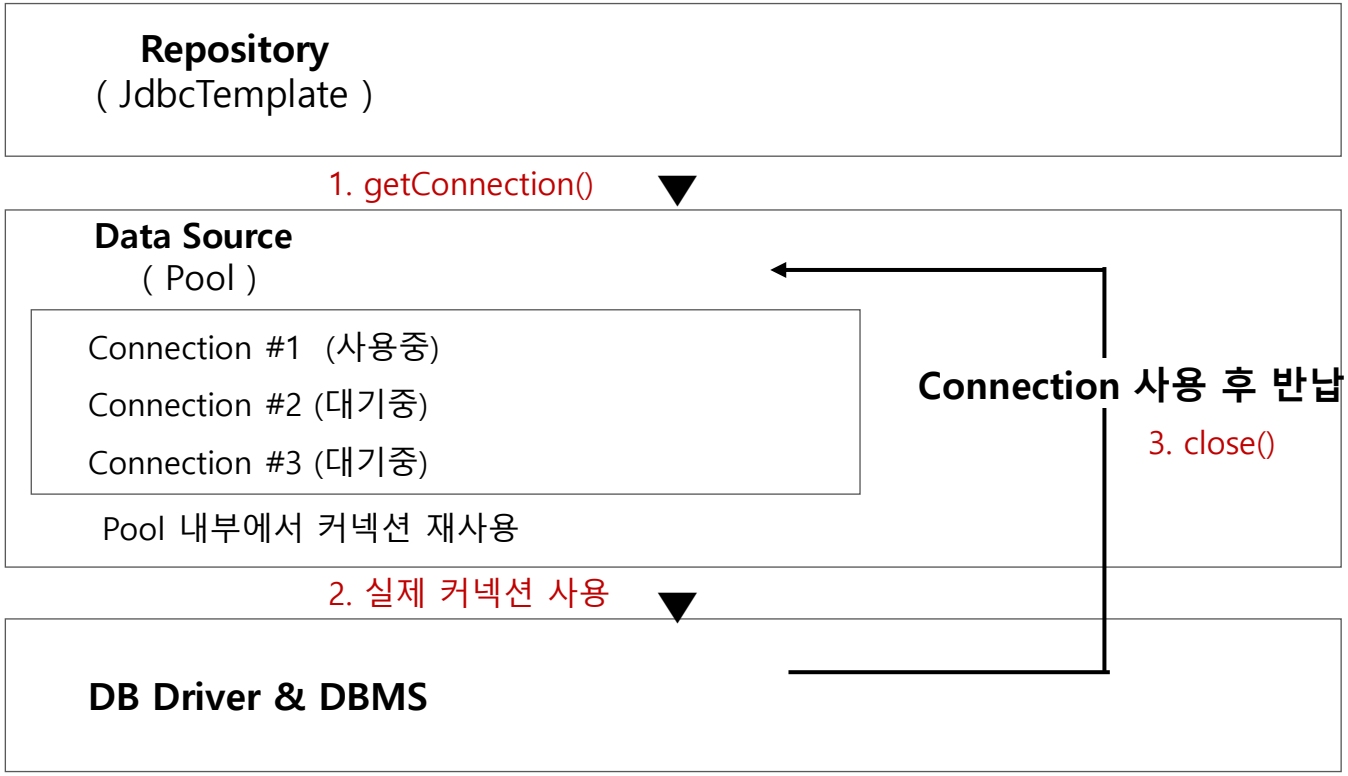
- 개념
  - 어플리케이션 시작 시 미리 일정 수의 Connection을 생성해서 풀(Pool)에 보관하고
  - 요청 시 풀(Pool)에서 빌려 쓰는 방식
  - DataSource 인터페이스를 통해서 Connection 관리 책임을 외부로 위임

- 장점
  - Connection 생성/종료 비용 없음
  - 응답 속도 및 서버 안정성 향상
  - 트랜잭션 관리 용이

- 대표 구현체
  - **HikariCP** (Spring Boot 3 기본)
  - DBCP2

4.1 데이터베이스 연동 기초

커넥션 풀(Connection Pool) 구조



4.1 데이터베이스 연동 기초

HikariCP 설정 예

```
spring:
  datasource:
  url: jdbc:mysql://localhost:3306/testdb
  username: root
  password: 1234
  driver-class-name: com.mysql.cj.jdbc.Driver
  hikari:
    maximum-pool-size: 10          # 풀의 최대 커넥션 수 (동시접속자 고려)
    minimum-idle: 5                # 최소 유휴 커넥션 수
    idle-timeout: 30000            # 유휴 커넥션 유지 시간(ms)
    max-lifetime: 1800000          # 커넥션의 최대 생존 시간(ms)
    connection-timeout: 30000      # 커넥션 요청 시 대기 시간(ms)
    pool-name: DemoHikariPool
```

4.1 데이터베이스 연동 기초

HikariCP + Actuator

- 활성화

```
management:
  endpoints:
    web:
      exposure:
        include: "health,metrics"
```

- 주요 지표

Metric	설명
hikaricp.connections	전체 커넥션 수
hikaricp.connections.active	사용 중 커넥션
hikaricp.connections.idle	대기 중 커넥션
hikaricp.connections.pending	커넥션 대기 요청 수

4.1 데이터베이스 연동 기초

JdbcTemplate

- 개념
  - Spring Framework가 제공하는 JDBC 코드 단순화 유틸리티 클래스
  - 반복되는 JDBC 코드인 보일러플레이트를 자동으로 처리하여 SQL 작성에만 집중 가능
- 의존성
  - implementation 'org.springframework.boot:spring-boot-starter-jdbc'
- 특징
  - 보일러플레이트 코드 자동화
  - 자동 자원 해제
  - 예외처리 변환 (SQLException 에서 DataAccessException 으로 변환)
  - 트랜잭션 연동 지원

4.1 데이터베이스 연동 기초

JdbcTemplate 주요 메서드

분류	메서드	설명
조회(SELECT)	query(sql, RowMapper<T> rm) queryForObject(sql, Class<T> type)	다중 행 조회 단일 행/컬럼 조회
DML(삽입/수정/삭제)	update(sql, Object ... args)	DML 실행 (영향받은 행 개수 반환)
ResultSet 매핑	RowMapper<T>	이전 ResultSet을 객체 변환 지원
트랜잭션 처리	@Transactional	Spring이 자동으로 commit/rollback처리
일괄 처리 (Batch)	batchUpdate(sql, List<Object[]> args)	다중 쿼리 일괄 실행

4.1 데이터베이스 연동 기초

JdbcTemplate 예

```
@Repository
public class UserRepository {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    // 전체 조회
    public List<User> findAll() {
        String sql = "SELECT id, name, email FROM users";
        return jdbcTemplate.query(sql, (rs, rowNum) ->
            new User(
                rs.getInt("id"),
                rs.getString("name"),
                rs.getString("email")
            )
        );
    }
```

```
// 단건 조회
public User findById(int id) {
    String sql = "SELECT * FROM users WHERE id = ?";
    return jdbcTemplate.queryForObject(sql,
            (rs, rowNum) -> new User(
                rs.getInt("id"),
                rs.getString("name"),
                rs.getString("email")
            ), id);
}
```

4.1 데이터베이스 연동 기초

JdbcTemplate 예

```
// 등록
    public int insert(User user) {
        String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
        return jdbcTemplate.update(sql, user.getName(), user.getEmail());
    }

    // 수정
    public int update(User user) {
        String sql = "UPDATE users SET name=?, email=? WHERE id=?";
        return jdbcTemplate.update(sql, user.getName(), user.getEmail(), user.getId());
    }

    // 삭제
    public int delete(int id) {
        String sql = "DELETE FROM users WHERE id = ?";
        return jdbcTemplate.update(sql, id);
    }
}
```

4.1 데이터베이스 연동 기초

트랜잭션 처리

- @Transactional 이용
  - Spring 의 선언적 트랜잭션 처리 방법
  - 성공하면 자동 commit, 예외 발생 시 자동 rollback 처리됨
  - 클래스 레벨 및 메서드 레벨 지원

ex.
```
@Service
public class UserService {

   @Autowired
   private UserRepository userRepository;

   @Transactional  // 메서드 레벨
   public void registerUser(User user) {
      userRepository.insert(user);
      userRepository.updateLog(user.getId(), "가입 완료");
   }

  public void deleteUser(User user){..}
  public void updateUser(Uer user){..}
}
```

ex.
```
@Service
@Transactional  // 클래스 레벨
public class UserService {

   @Autowired
   private UserRepository userRepository;

   public void registerUser(User user) {
      userRepository.insert(user);
      userRepository.updateLog(user.getId(), "가입 완료");
   }

  public void deleteUser(User user){..}
  public void updateUser(Uer user){..}
}
```

4.1 데이터베이스 연동 기초

비교 요약

구분	기본 JDBC	Spring JDBC (JdbcTemplate)
패키지	java.sql	org.springframework.jdbc.core
커넥션 관리	수동	자동 (DataSource + Pool)
SQL 실행 방식	직접	Template API ex. query, queryForObject, update
자원 해제	수동 close()	자동 처리
예외 처리	예외처리 필수 (SQLException)	예외처리 불필요(DataAccessException, 런타임)
트랜잭션	수동 commit/rollback	@Transactional 이용한 선언적 방법
코드량	중복코드 많음	간결하고 유지보수 쉬움
성능	연결 비용 큼	커넥션 풀로 효율적
결론	JDBC 기반 기술 의미	JDBC 단점 보완한 고수준 API, 실무형 개발용

4.2 MyBatis 연동

MyBatis

- 개념
  - SQL 중심의 퍼시스턴스 프레임워크로서 SQL은 직접 작성하고 파라미터 바인딩 및
  - 결과 매핑을 자동화함
  - ORM(Hibernate/JPA)처럼 객체-관계 매핑을 광범위하게 위임하지 않고 쿼리 주도 설계(Query-Driven)로 세밀한 제어가 가능

- 의존성
  - **implementation 'org.mybatis.spring.boot:mybatis-spring-boot-starter:3.0.3'**

4.2 MyBatis 연동

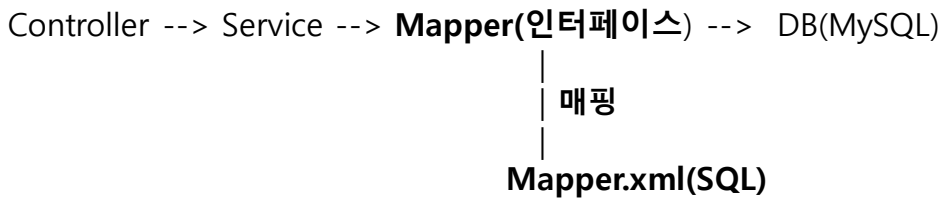
MyBatis

- 특징
  - SQL과 Java 객체 간 직접 매핑 지원
  - SQL을 직접 제어하면서도 기존 JDBC 코드의 보일러플레이트 코드 제거
  - SQL 작성 자유도 및 가독성 높음 (SQL 친화적 팀에 적합)
  - 동적 SQL 지원 ex. if, choose, where, foreach, trim, set
  - 정교한 결과 매핑 ex. resultMap, association, collection
  - XML 및 어노테이션 2가지 작성 방식 지원

4.2 MyBatis 연동

MyBatis 구성요소

구성요소	설명
SqlSessionFactory / SqlSession	MyBatis와 DB 연결 관리 (기존 Connection 역할)
Mapper XML	실제 SQL문이 작성되는 XML 파일
Mapper Interface	Mapper XML내의 SQL문과 매핑하기 위한 인터페이스 @Mapper 지정 (기존 Repository 역할)
DTO/VO	데이터 전송 객체 (자바와 DB 간 자동 매핑)
Service	비즈니스 로직 및 트랜잭션 담당 @Service, @Transactional 지정
Controller	요청 및 응답 처리, @Controller



4.2 MyBatis 연동

Mapper XML

- 개념
  - MyBatis에서 SQL문을 작성/관리하고, SQL문과 Java 객체간 매핑 규칙을 기술하는 XML 파일
  - 저장 경로는 관례적으로 src/main/resources/mappers/*.xml 사용

- 매핑 원리

Mapper Interface	Mapper XML
Mapper 인터페이스의 FQCN (패키지 포함 전체 이름) ex. **com.example.demo.mappers.EmpMapper**	namespace 값 ex. **namespace="com.example.demo.mappers.EmpMapper"**
Mapper 인터페이스의 메서드명 ex. public void **add**(...); public List<String> **list**();	<select><insert> 태그 등의 id 속성값 ex. <insert **id**="**add**" >, <select **id**="**list**" >
Mapper 인터페이스의 메서드 파라미터 타입 ex. public void add(**User** user);	<select><insert> 태그 등의 parameterType 속성값 ex. <insert id="add" **parameter="com.exam.User">**
Mapper 인터페이스의 메서드 리턴 타입 ex. public **User** findById(String id);	<select> 태그의 resultType 속성값 ex. <select id="findById" parameter="string" **resultType="com.exam.User"** >

4.2 MyBatis 연동

Mapper XML 및 Interface 기본 골격

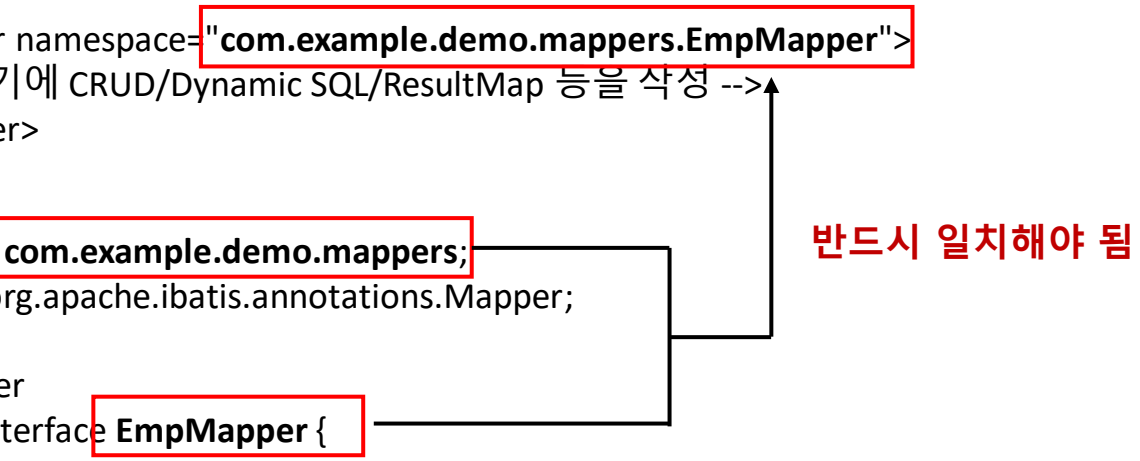
```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
  PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
  "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.example.demo.mappers.EmpMapper">
  <!-- 여기에 CRUD/Dynamic SQL/ResultMap 등을 작성 -->
</mapper>


package com.example.demo.mappers;
import org.apache.ibatis.annotations.Mapper;

@Mapper
public interface EmpMapper {

}
```



4.2 MyBatis 연동

CRUD + α 기본 문법

기능	태그	파라미터 값 전달	리턴값 반환
조회	<select id="">	parameterType	resultType / resultMap
추가	<insert id="">	parameterType	영향 받은 레코드 갯수
수정	<update id="">	parameterType	영향 받은 레코드 갯수
삭제	<delete id="">	parameterType	영향 받은 레코드 갯수
조건	<if>	test 속성으로 조건지정	-
다중조건	<choose>,<when>,<otherwise>	test 속성으로 조건 분기	-
반복문	<foreach>	collection 반복	-
SQL 재사용	<sql>, <include>	-	-
동적 SQL 조립	<where>	내부 if 문과 함께 사용	-
매핑	<association>, <collection>	1:1 연관 매핑, 1:N 연관 매핑	-

4.2 MyBatis 연동

SELECT 단일 처리 매핑

```
<select id="findById"
parameterType="int"
resultType="com.example.demo.dto.EmpDTO">
  SELECT empno, ename, job, sal, deptno
  FROM emp
  WHERE empno = #{empno}
</select>
```

```
@Mapper
public interface EmpMapper {

    EmpDTO findById(int empno);
}
```

resultType 속성값

parameterType

id 속성값

parameter 값
#{empno} 치환됨

4.2 MyBatis 연동

SELECT 단일 처리 매커니즘

```
<select id="findById"
parameterType="int"
resultType="com.example.demo.dto.EmpDTO">
  SELECT empno, ename, job, sal, deptno
  FROM emp
  WHERE empno = #{empno}
</select>
```

자동저장

자동 매핑 방법은 실행결과의 컬럼헤딩에 해당하는 레코드값을 EmpDTO의 setter메서드에 전달해서 저장시킴.

ex. ename 컬럼헤딩값은 set**Ename**(String ename){...} 에 전달됨
sal 컬럼헤딩값은 set**Sal**(Double sal){....}에 전달됨
만약 setSalary(Double sal){..} 였다면 전달 안됨.

setEmpno 호출
7369 전달

setSal 호출
800 전달

1 empno	2 ename	3 job	4 sal	5 deptno
7,369	SMITH	CLERK	800	20

4.2 MyBatis 연동

SELECT 다중 처리 매핑

```
<select id="findAll"
resultType="com.example.emp.dto.EmpDTO">
  SELECT empno, ename, job, sal, deptno
  FROM emp
  ORDER BY empno
</select>
```

```
@Mapper
public interface EmpMapper {

   List<EmpDTO> findAll();
}
```

resultType 속성값

id 속성값

복수행

4.2 MyBatis 연동

INSERT 처리 매핑

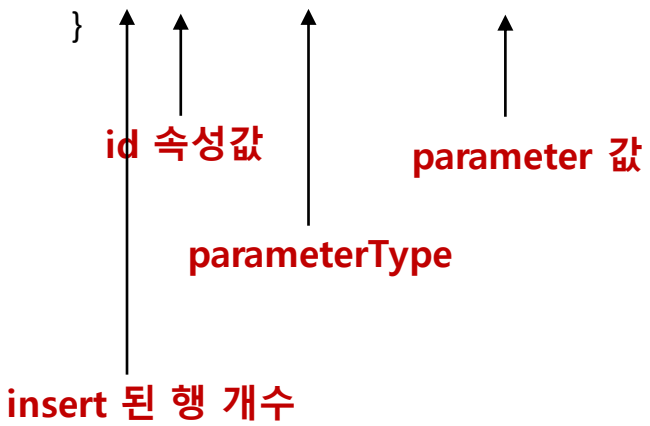
```
<insert id="insert"
parameterType="com.example.emp.dto.EmpDTO">
  INSERT INTO emp (empno, ename, job, sal, deptno)
  VALUES (#{empno}, #{ename}, #{job}, #{sal}, #{deptno})
</insert>
```

AUTO_INCREMENT 이용
(반드시 스키마에서 **AUTO_INCREMENT로 되어 있어야 됨)**

```
<insert id="insert"
parameterType="com.example.emp.dto.EmpDTO“
useGeneratedKeys="true" keyProperty="empno">
  INSERT INTO emp ( ename, job, sal, deptno)
  VALUES ( #{ename}, #{job}, #{sal}, #{deptno})
</insert>
```

```
@Mapper
public interface EmpMapper {

   int insert(EmpDTO empDTO);
}
```



4.2 MyBatis 연동

INSERT 처리 매커니즘

```
<insert id="insert"
parameterType="com.example.emp.dto.EmpDTO">
  INSERT INTO emp (empno, ename, job, sal, deptno)
  VALUES (#{empno}, #{ename}, #{job}, #{sal}, #{deptno})
</insert>
```

자동 설정

자동 매핑방법은 지정된 바인드 변수(#{변수})와 일치하는 EmpDTO의 getter메서드가 호출됨.

ex. #{**ename**} 은 get**Ename**(){...} 리턴값으로 치환
#{**sal**} 은 get**Sal**(){...} 리턴값으로 치환

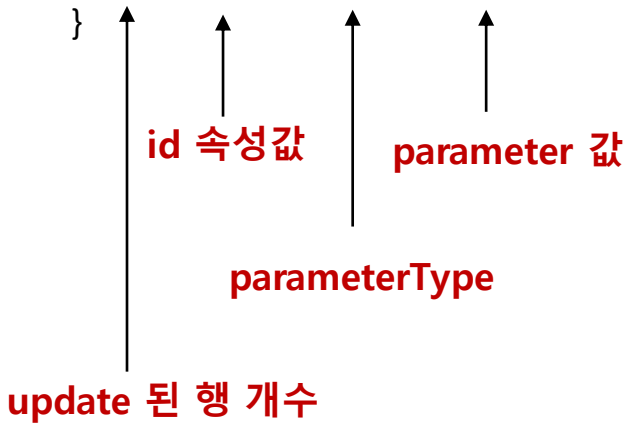
4.2 MyBatis 연동

UPDATE 처리

```
<update id="update"
parameterType=" com.example.emp.dto.EmpDTO ">
  UPDATE emp
  SET ename=#{ename}, job=#{job}, sal=#{sal},
     deptno=#{deptno}
  WHERE empno=#{empno}
</update>
```

```
@Mapper
public interface EmpMapper {

   int update(EmpDTO empDTO);
}
```



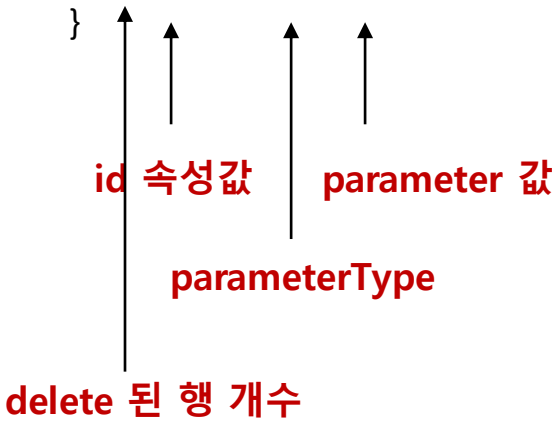
4.2 MyBatis 연동

DELETE 처리

```
<delete id="delete" parameterType="int">
  DELETE FROM emp WHERE empno=#{empno}
</delete>
```

```
@Mapper
public interface EmpMapper {

   int delete(int empno);
}
```



4.2 MyBatis 연동

재사용 가능한 SQL 조각

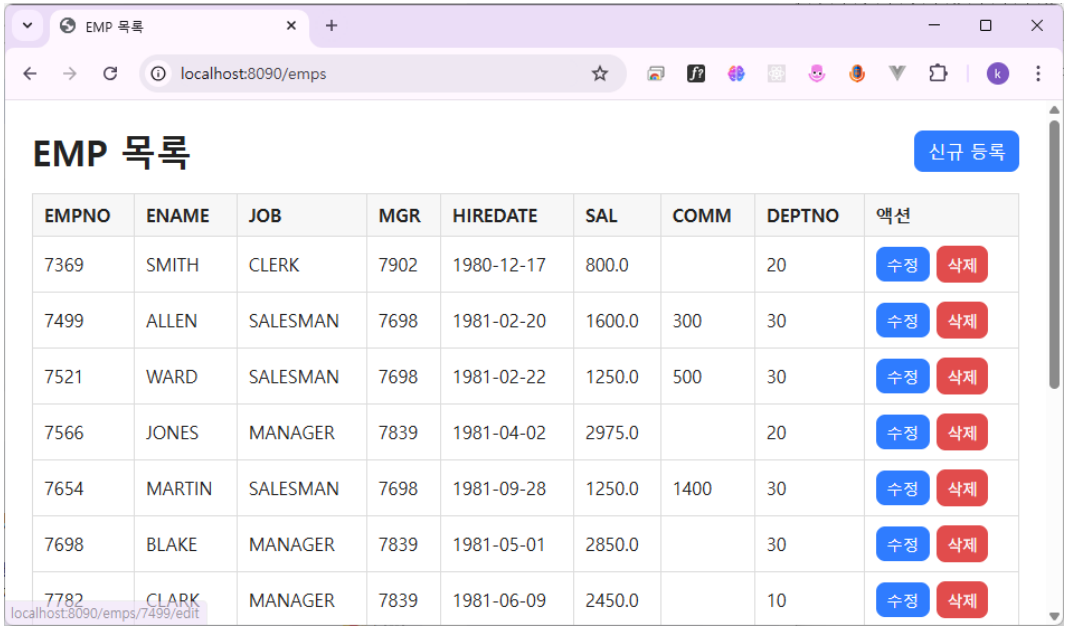
```
<!-- 공통 컬럼 목록 -->
<sql id="emp_columns">
  empno, ename, job, sal, deptno
</sql>

<select id="findAllReuse" resultType="EmpDTO">
  SELECT <include refid="emp_columns"/>
  FROM emp
</select>
```

치환됨

4.2 MyBatis 연동

실습(MyBatis)




```
# build.gradle
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    implementation 'org.mybatis.spring.boot:mybatis-spring-boot-starter:3.0.3'
     runtimeOnly 'com.mysql:mysql-connector-j'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

4.2 MyBatis 연동

실습 프로젝트 구조 (MyBatis)

```
src/main/java/
    com/example/demo
                DemoApplication.java                    # @SpringBootApplication
                 controller/EmpController.java          # @Controller
                 dto/EmpDTO.java                        # Emp 테이블 DTO
                 mappers/
                     EmpMapper.java                     # 매핑 인터페이스, @Mapper, @Param
                  service/
                      EmpService.java
                      EmpServiceImpl.java               # @Service, @Transactional
src/main/resources
     static/css/style.css
     templates/add.html, edit.html, list.html           # Thymeleaf 템플릿
     mappers/EmpMapper.xm                               # SQL 문 포함
     application.yml                                    # Spring MyBatis 연동 설정
```

4.2 MyBatis 연동

자동 매핑 (Auto Mapping)

- 개념
  - SQL 결과(ResultSet)의 컬럼 이름과 DTO의 필드 이름이 일치하면 자동으로 값을 설정

- 작동방식
  - 내부적으로 두 단계로 자동 매핑을 수행함

단계	설명
1단계	컬럼 이름과 필드 이름이 일치하면 자동 매핑 ex. empno 컬럼값이 empno 필드에 자동 매핑
2단계	설정에 따라서 언더스코어(_) 컬럼이 카멜표기 필드에 변환되어 매핑 ex. emp_no 컬럼값이 empNo 필드에 자동 매핑

```
mybatis:
  configuration:
    map-underscore-to-camel-case: true     # 언더스코어 자동 매핑
```

4.2 MyBatis 연동

명시적 매핑 (<resultMap>)

- 개념
  - SQL 결과(ResultSet)를 Java 객체(DTO)에 명시적으로 매핑해주는 설정 태그

- 구성요소

요소	설명
<resultMap>	전체 매핑 시작. id 속성: 이 매핑의 이름 (다른 SQL에서 참조 가능) type 속성: 결과를 매핑할 DTO 클래스의 전체 경로 (FQCN)
<id>	기본키(Primary key) 컬럼 지정 column 속성: SQL 결과 컬럼명 property 속성: DTO 필드명
<result>	일반 컬럼과 DTO 필드 매핑 column 속성: SQL 결과 컬럼명 property 속성: DTO 필드명

4.2 MyBatis 연동

<resultMap> 예시

```
<resultMap id="EmpResultMap" type="com.example.demo.dto.EmpDTO">
  <id    property="empno"    column="empno"/>
  <result property="ename"    column="ename"/>
  <result property="job"      column="job"/>
  <result property="mgr"      column="mgr"/>
   <result property="hiredate"     column="hiredate"/>
</resultMap>

<select id="findAll" resultMap="EmpResultMap">          # 다른 SQL에서 참조 가능
  select
     e.empno,
     upper(e.ename)        as ename,                    -- 대문자로 변환
     concat(e.job, ' 사원')  as job,                     -- 문자열 결합
     ifnull(e.mgr, 0)      as mgr,                      -- null 처리
     date_format(e.hiredate, '%Y-%m-%d') as hiredate,   -- 날짜 포맷팅
  from emp e
  order by e.empno
</select>
```

4.2 MyBatis 연동

관계 매핑

- 개념
  - 데이터베이스 테이블 간의 연관 관계(relationship)를 자바 객체(DTO)간의
  - 객체 관계(Object relationship)로 연결해주는 매핑 문법
  - 즉 DB의 조인(join) 결과를 하나의 자바 객체 내부 구조로 표현하는 기술임

- 매핑 종류

관계	설명	MyBatis 태그	예
1:1 (has-one)	한 객체가 다른 객체 하나를 포함	<association>	Emp 안에 Dept
1:N (has-many)	한 객체가 다른 객체 여러 개를 포함	<collection>	Dept 안에 List<Emp>

4.2 MyBatis 연동

<association>

- 개념
  - 1:1 또는 N:1 관계로서 자바 객체 안에 다른 객체 1개를 포함할 때 사용
    ex. 한 직원이 속한 부서 한 곳

- 자바 코드 표현

```
public class EmpDTO {

  private int empno;
  private String ename;
  private String job;
  private int mgr;
  private String hiredate;

  private DeptDTO dept;  // has-one 관계
```

4.2 MyBatis 연동

<association>

▪ 문법

속성	설명
property	상위 객체(EmpDTO)의 필드명 (getter/setter 기반)
javaType	하위 객체(DeptDTO)의 클래스 타입
내부 <id>/<result>	하위 객체(DeptDTO) 컬럼 매핑

4.2 MyBatis 연동

<association> 예시

```
<resultMap id="EmpWithDeptMap"
type="com.example.demo.dto.EmpDTO">
 <id property="empno" column="empno"/>
 <result property="ename" column="ename"/>
 <result property="job" column="job"/>
 <result property="deptno" column="deptno"/>

 <!-- N:1 관계 (has-one) -->
 <association property="dept"
              javaType="com.example.demo.dto.DeptDTO">
  <id property="deptno" column="d_deptno"/>
  <result property="dname" column="d_dname"/>
  <result property="loc" column="d_loc"/>
 </association>

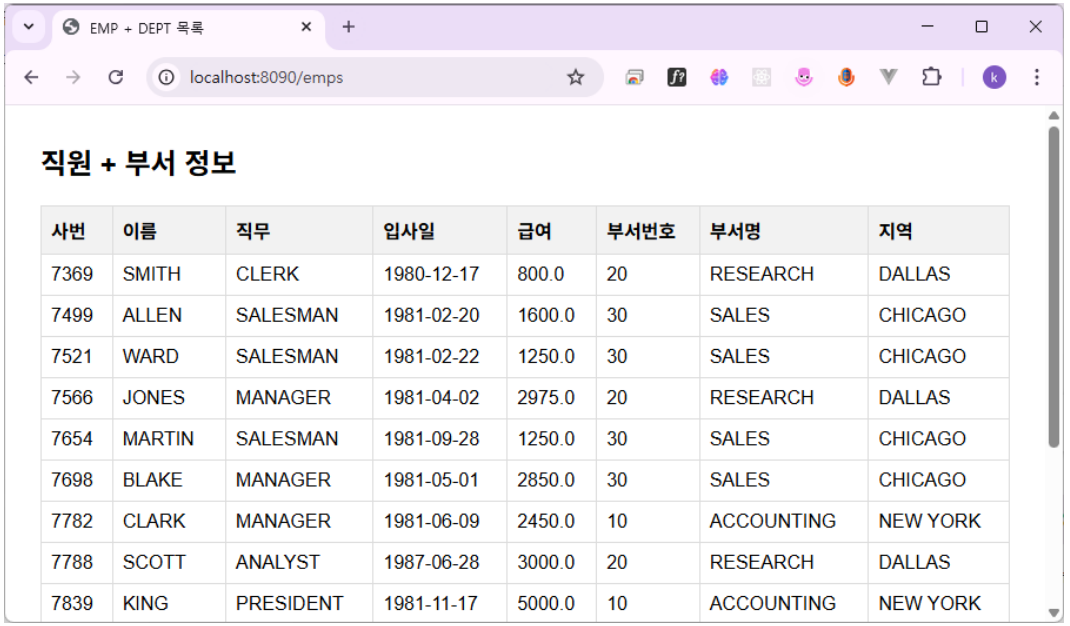
</resultMap>
```

```
<select id="findAllWithDept" resultMap="EmpWithDeptMap">
 select e.empno, e.ename, e.job, e.deptno,
        d.deptno as d_deptno,
        d.dname as d_dname,
        d.loc as d_loc
 from emp e  join dept d on e.deptno = d.deptno
</select>
```

참조

4.2 MyBatis 연동

실습(<association>)



4.2 MyBatis 연동

<collection>

- 개념
  - 1:N 관계로서 자바 객체 안에 다른 객체들의 리스트(List)를 포함할 때 사용
    ex. 한 부서에 속한 여러 직원

- 자바 코드 표현

```
class DeptDTO {
  private Integer deptno;
  private String dname;

  private List<EmpDTO> employees;  // has-many 관계
}
```

4.2 MyBatis 연동

< collection >

▪ 문법

속성	설명
property	상위 객체(DeptDTO)의 리스트 필드명 ex. List<EmpDTO> employees
Of Type	리스트 내부 객체의 타입 ex. List<EmpDTO> employees

4.2 MyBatis 연동

<collection> 예시

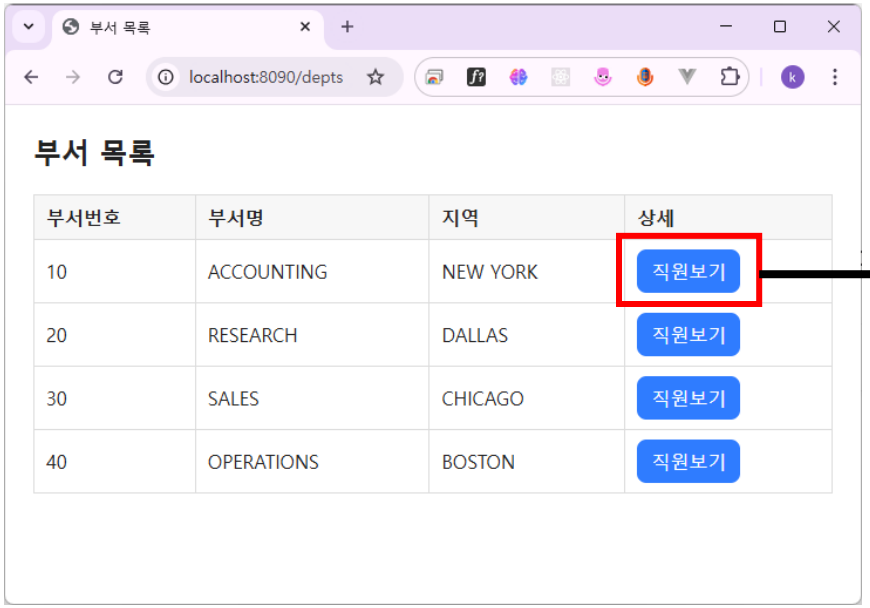
```
<resultMap id="emp_dept_join" type="DeptDTO">
  <id property="deptno" column="deptno" />
  <result property="dname" column="dname" />
  <result property="loc" column="loc" />

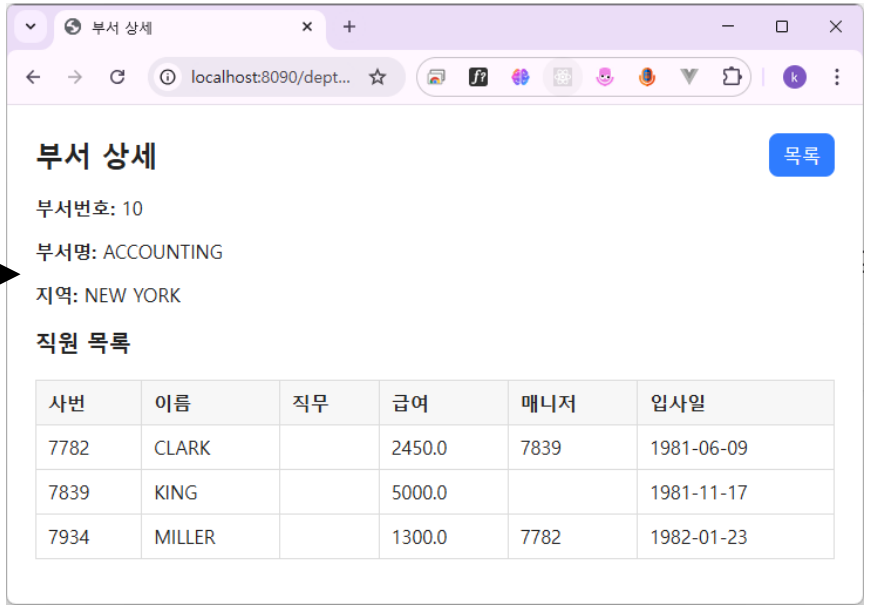
  <collection  property="employees" ofType="EmpDTO" >
    <result column="empno"  property="empno"/>
    <result column="ename"  property="ename"/>
    <result column="sal" property="sal"/>
    <result column="mgr" property="mgr"/>
    <result column="hiredate" property="hiredate"/>
  </collection>
</resultMap>

<select id="empDeptjoin" resultMap="emp_dept_join">
      select *
      from emp join dept on emp.deptno = dept.deptno
     order by 1
</select>
```

4.2 MyBatis 연동

실습(<collection>)





4.2 MyBatis 연동

동적 SQL

- 개념
  - 프로그램 실행 중 조건에 따라서 SQL문이 동적으로 생성되는 기능
  - if, choose, foreach 등의 XML 태그로 SQL문의 일부를 제어

- 주요 태그

태그명	설명	주요 용도
<if> / <where>	조건 / where 자동 추가 및 조건 정리	선택적 조건 추가 / 검색조건
<choose><when> <otherwise>	if-else 구조	조건 분기
<trim>	SQL의 불필요한 AND/OR 제거	WHERE 절 구성
<set>	UPDATE 시 SET 절 자동 구성	UPDATE
<foreach>	컬렉션 반복	다중 INSERT, IN 조건

4.2 MyBatis 연동

<where> + <if>

- 개념
  - 여러 검색 조건이 있을 때, 값이 존재하는 것만 WHERE 절에 자동으로 추가

항목	설명
<where>	SQL 문장에 WHERE를 자동으로 추가하고 불필요한 AND/OR를 정리해 줌
<if test=조건>	특정 조건이 true인 경우에만 SQL문을 포함시킴
조합 효과	값이 null이거나 비어 있으면 해당 조건을 SQL에서 자동 생략

- 예시

  검색폼에서 ename 또는 deptno 로 조회하고 싶은 경우
  - 둘 다 입력하면 두 조건 모두 적용
  - 하나만 입력하면 그 조건만 적용
  - 아무것도 입력 안 하면 전제 조회

4.2 MyBatis 연동

<where> + <if> 예시

```
<select id="searchEmp"
        parameterType="com.example.demo.dto.EmpSearchCond"
        resultType="com.example.demo.dto.EmpDTO">
  SELECT empno, ename, job, sal, deptno
  FROM emp
  <where>
    <if test="ename != null and ename != ''">
      ename LIKE CONCAT('%', #{ename}, '%')
    </if>
    <if test="deptno != null">
      AND deptno = #{deptno}
    </if>
  </where>
  ORDER BY empno
</select>
```

```
@Mapper
public interface EmpMapper {
   List<EmpDTO> searchEmp(EmpSearchCond  cond);
}
```

```
public class EmpSearchCond {
    private String ename;   // 사원 이름
    private Integer deptno; // 부서번호
}
```

4.2 MyBatis 연동

<choose>

- 개념
  - 자바의 if-else if- else 구문과 동일한 역할을 수행하는 조건 분기 태그 (상호 배타)

항목	설명
<choose>	조건 분기의 시작
<when>	각 조건을 표현
<otherwise>	모든 조건이 만족하지 않을 때

- 기본 문법

```
<choose>
  <when test="조건1"> ... </when>
  <when test="조건2"> ... </when>
  <otherwise> ... </otherwise>
</choose>
```

4.2 MyBatis 연동

<choose> 예시

```
<select id="searchByMode" parameterType="map" resultType="EmpDTO">
  SELECT empno, ename, sal, deptno
  FROM emp
  <where>
    <choose>
      <when test="mode == 'ename' and kw != null and kw != ''">
        ename LIKE CONCAT('%', #{kw}, '%')
      </when>
      <when test="mode == 'deptno' and deptno != null">
        deptno = #{deptno}
      </when>
      <otherwise>
        1 = 1  <!-- 기본: 전체 -->
      </otherwise>
    </choose>
  </where>
</select>
```

4.2 MyBatis 연동

<set>

- 개념
  - SQL의 UPDATE 문에서 SET 절을 동적으로 구성할 때 사용하는 태그
  - <set> 내부에 <if>와 같이 사용하여 값이 존재하는 컬럼만 수정하도록 SQL 작성
  - 자동으로 마지막 콤마(,)를 제거함

- 기본 문법

```
UPDATE emp
 <set>
  <if test="조건"> 컬럼1=값, </if>
  <if test="조건"> 컬럼2=값, </if>
  <if test="조건"> 컬럼3=값, </if>
 </set>
 WHERE 컬럼4=값
```

4.2 MyBatis 연동

<set> 예시

```
<update id="updateEmp" parameterType="EmpDTO">
  UPDATE emp
  <set>
    <if test="ename != null"> ename = #{ename}, </if>
    <if test="job != null"> job = #{job}, </if>
    <if test="sal != null"> sal = #{sal}, </if>
    <if test="deptno != null"> deptno = #{deptno}, </if>
  </set>
  WHERE empno = #{empno}
</update>
```

4.2 MyBatis 연동

<foreach>

- 개념
  - 컬렉션(List, Set, Array, Map 등) 형태의 데이터를 SQL문에 반복적으로 삽입하거나 조건절을 구성할 때 사용하는 태그

- 기본 문법

```
<foreach
 collection="컬렉션명"
 item="각요소명"
 index="인덱스명(선택)"
 open="앞에 붙일 문자"
 close="뒤에 붙일 문자"
 separator="요소 사이 구분자">
 #{item}
</foreach>
```

4.2 MyBatis 연동

<foreach> 예시1 – IN 조건문

목표 SQL

SELECT * FROM emp WHERE deptno **IN (10, 20, 30)**

Mapper XML

```
<select id="findByDeptList" resultType="EmpDTO" parameteType="arraylist">
  SELECT empno, ename, job, sal, deptno
  FROM emp
  WHERE deptno IN
        <foreach collection="deptList" item="dept" open="(" separator="," close=")">
          #{dept}
        </foreach>
</select>
```

Mapper Interface

```
List<EmpDTO> findByDeptList(@Param("deptList") List<Integer> deptList);
```

호출 예시

```
List<Integer> list = Arrays.asList(10, 20, 30);
List<EmpDTO> result = empMapper.findByDeptList(list);
```

4.2 MyBatis 연동

<foreach> 예시2 – 다중 INSERT

목표 SQL

```
INSERT INTO emp (empno, ename, job, sal, deptno) VALUES
(1001,'A','SALESMAN',1000,10),(1002,'B','CLERK',1200,20);
```

Mapper XML

```
<insert id="insertBatch" parameterType="arraylist">
  INSERT INTO emp (empno, ename, job, sal, deptno)
  VALUES
  <foreach collection="list" item="e" separator=",">
    (#{e.empno}, #{e.ename}, #{e.job}, #{e.sal}, #{e.deptno})
  </foreach>
</insert>
```

Mapper Interface

```
int insertBatch(@Param("list") List<EmpDTO> list);
```

호출 예시

```
List<EmpDTO> emps = new ArrayList<>();
emps.add(new EmpDTO(1001, "A", "SALESMAN", 1000, 10));
emps.add(new EmpDTO(1002, "B", "CLERK", 1200, 20));
empMapper.insertBatch(emps);
```

4.2 MyBatis 연동

<foreach> 예시3 – Map 내 List

목표 SQL

```
SELECT * FROM emp WHERE job IN ('CLRK','SALESMAN') AND deptno IN (10, 20);
```

Mapper XML

```
<select id="findByJobAndDept" resultType="EmpDTO" parameterType="hashmap">
 SELECT * FROM emp
 WHERE job IN
 <foreach collection="map.jobList" item="j" open="(" separator="," close=")">
  #{j}
 </foreach>
 AND deptno IN
 <foreach collection="map.deptList" item="d" open="(" separator="," close=")">
  #{d}
 </foreach>
</select>
```

Mapper Interface

```
List<EmpDTO> findByJobAndDept(@Param("map")
                              Map<String, Object> map);
```

호출 예시

```
Map<String, Object> map = new HashMap<>();
map.put("jobList", Arrays.asList("CLERK", "SALESMAN"));
map.put("deptList", Arrays.asList(10, 20));
empMapper.findByJobAndDept(map);
```

4.2 MyBatis 연동

<trim>

- 개념
  - 동적 SQL에서 앞/뒤에 불필요하게 붙는 토큰(AND, OR, 콤마 등)을 자동으로 정리해 주는 범용 유틸리티 태그
  - <where><set>이 각각 WHERE/SET 절에 특화된 태그라면 <trim>은 더 범용적으로 앞뒤에 붙일 문자열을 제어하고 지정한 토큰을 제거 할 수 있음

- 주요 속성

속성	역할	예시
prefix	<trim> 블록 앞에 자동으로 붙일 문자열	prefix="WHERE" / prefix="SET"
prefixOverrides	블록 맨 앞에 존재하면 제거할 토큰목록	prefixOverrides="AND | OR"
suffix	블록 뒤에 자동으로 붙일 문자열	suffix=")"
suffixOverrides	블록 맨 뒤에 존재하면 제거할 토큰목록	suffixOverrides=","

4.2 MyBatis 연동

<trim> 예시1 – WHERE 절 동적 구성

- <where> 대신 <trim prefix="WHERE" prefixOverrides="AND | OR">를 사용
- 첫 번째 AND/OR 제거

```
<select id="searchWithTrim"
      parameterType="com.example.emp.dto.EmpSearchCond"
      resultType="com.example.emp.dto.EmpDTO">
  SELECT empno, ename, job, sal, deptno
  FROM emp
  <trim prefix="WHERE" prefixOverrides="AND | OR">
    <if test="ename != null and ename != ''">
      AND ename LIKE CONCAT('%', #{ename}, '%')
    </if>
    <if test="deptno != null">
      AND deptno = #{deptno}
    </if>
  </trim>
  ORDER BY empno
</select>
```

4.2 MyBatis 연동

<trim> 예시2 – SET 절 동적 구성

- <set> 대신 <trim prefix="SET" suffixOverrides=",">로 마지막 콤마 제거

```
<update id="updateDynamic" parameterType="EmpDTO">
  UPDATE emp
  <trim prefix="SET" suffixOverrides=",">
    <if test="ename != null"> ename = #{ename}, </if>
    <if test="job != null"> job = #{job}, </if>
    <if test="sal != null"> sal = #{sal}, </if>
    <if test="deptno != null"> deptno = #{deptno}, </if>
  </trim>
  WHERE empno = #{empno}
</update>
```

4.2 MyBatis 연동

<trim> 예시3 – INERT문 컬럼/VALUES 동적 구성

- 선택 입력 필드만 컬럼/값에 포함
  (컬럼과 값 쌍이 동일한 순서/개수로 유지되도록 주의)

```
<insert id="insertDynamic" parameterType="EmpDTO">
 INSERT INTO emp
 <trim prefix="(" suffix=")" suffixOverrides=",">
  <if test="empno != null"> empno, </if>
  <if test="ename != null"> ename, </if>
  <if test="job != null"> job, </if>
 </trim>
 VALUES
 <trim prefix="(" suffix=")" suffixOverrides=",">
  <if test="empno != null"> #{empno}, </if>
  <if test="ename != null"> #{ename}, </if>
  <if test="job != null"> #{job}, </if>
 </trim>
</insert>
```

4.2 MyBatis 연동

어노테이션 (Annotations)

- 개념
  - Mapper 인터페이스 메서드에 직접 SQL을 지정하여 사용하는 방식
  - Mapper XML 파일 없이도 SQL을 정의하고 실행 가능

- 특징

구분	설명
장점	XML 없이 빠르게 CRUD 구현, 유지보수 편리(소규모), 코드와 밀접
단점	복잡한 SQL 시 가동성 저하, SQL 재사용 어려움, 조인 매핑 시 한계
추천 용도	간단 CRUD / 작은 프로젝트 / 프로토타입 개발
비추천 용도	복잡 동적 SQL, 다중 조인, SQL 재사용 많은 프로젝트

4.2 MyBatis 연동

어노테이션 (Annotations) 기본 문법

종류	역할	설명	예시
@Select	조회(SELECT) SQL 실행	단순 조회 시 작성 SQL 문자열 직접 작성	@Select("SELECT * FROM emp") List<EmpDTO> findAll();
@Insert	등록(INSERT) SQL 실행	새 레코드 추가 AUTO_INCREMENT 가능	@Insert("INSERT INTO emp(ename,sal) VALUES(#{ename},#{sal})")
@Update	수정(UPDATE) SQL 실행	기존 데이터 수정	@Update("UPDATE emp SET sal=#{sal} WHERE empno=#{empno}")
@Delete	삭제(DELETE) SQL 실행	레코드 삭제	@Delete("DELETE FROM emp WHERE empno=#{id}")
@Param	SQL 내에서 사용할 파라미터 이름 지정	파라미터 2개 이상일 때 추천, 내부적으로 Map 으로 처리	@Select("SELECT * FROM emp WHERE deptno=#{dno}") EmpDTO find(@Param("dno") int deptno);
@Results	SQL 결과를 객체 필드로 매핑	XML의 <resultMap> 과 동일 기능	@Results(id="EmpMap", value={ @Result(column="EMPNO",property="empno",id=true) })

4.2 MyBatis 연동

어노테이션 (Annotations) 예시

```
@Select("SELECT empno, ename, sal FROM emp ORDER BY empno")
List<EmpDTO> findAll();

@Select("SELECT empno, ename, sal FROM emp WHERE empno = #{id}")
EmpDTO findById(@Param("id") int empno);

@Select("SELECT EMPNO, ENAME, SAL, DEPTNO FROM EMP WHERE EMPNO = #{id}")
 @Results(id="EmpMap", value={
 @Result(column="EMPNO",  property="empno",  id=true),
 @Result(column="ENAME",  property="ename"),
 @Result(column="DEPTNO", property="deptno")
})
EmpDTO findWithMap(@Param("id") int id);

@Select("SELECT EMPNO, ENAME, SAL, DEPTNO FROM EMP")
@ResultMap("EmpMap") // 재사용
List<EmpDTO> findAllWithMap();
```

4.2 MyBatis 연동

어노테이션 (Annotations) 예시

```
@Insert("""
  INSERT INTO emp (ename, sal, deptno) VALUES (#{ename}, #{sal}, #{deptno})
""")
@Options(useGeneratedKeys = true, keyProperty = "empno") // AUTO_INCREMENT 컬럼 DTO에 반영
int insert(EmpDTO e);

@Update("UPDATE emp SET ename=#{ename}, sal=#{sal}, deptno=#{deptno} WHERE empno=#{empno}")
int update(EmpDTO e);

@Delete("DELETE FROM emp WHERE empno=#{id}")
int delete(@Param("id") int empno);
```

4.3 Lombok

Lombok

- 개념
  - Java의 반복적인 코드를 자동으로 생성해주는 컴파일 타임 어노테이션 처리기
  - 개발자가 작성해야 될 보일러플레이트 코드 등을 자동으로 생성함
    ex. getter/setter, toString(), equals(), hashCode(), 생성자, Builder

- 특징

항목	설명
주요 사용처	DTO, Entity, VO, Record 대체용 클래스
주요 목적	반복적인 코드 자동 생성
사용 시점	컴파일 시점
대표 어노테이션	@Getter, @Setter, @ToString, @NoArgsConstructor, @AllArgsConstructor, @Builder 등

4.3 Lombok

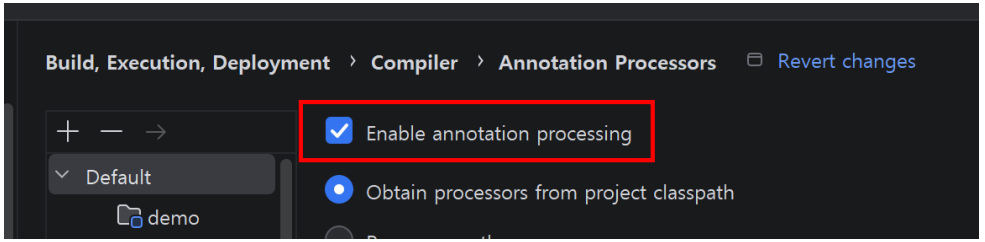
Lombok

- 의존성

```
dependencies {
   compileOnly 'org.projectlombok:lombok'
   annotationProcessor 'org.projectlombok:lombok'
   testCompileOnly 'org.projectlombok:lombok'
   testAnnotationProcessor  'org.projectlombok:lombok'
}
```

- Intellij IDE의 Lombok 활성화

settings > Build, Execution, Deployment > Compiler > Annotation Processors 에서 Enable annotation processing 체크 필요.



4.3 Lombok

Lombok 주요 어노테이션

어노테이션	설명
@Getter / @Setter	모든 필드의 getter/setter 메서드 자동 생성
@ToString	객체 정보를 문자열로 표현하는 toString() 메서드 자동 생성
@EqualsAndHashCode	equals() 와 hashCode() 자동 생성
@NoArgsConstructor	매개변수 없는 기본 생성자 생성
@AllArgsConstructor	모든 필드를 매개변수로 받는 생성자 생성
@RequiredArgsConstructor	final 또는 @NotNull 필드만 포함한 생성자 생성
@Builder	객체를 빌더 패턴으로 생성 가능 컬렉션 필드에는 @Builder.Default 지정하여 자동으로 초기화 지원
@Data	@Getter, @Setter,@ToString,@EqualsAndHashCode, @RequiredArgsConstructor 포함 (DTO 적합, Entity 지양)
@Slf4j	로깅용 Logger 자동생성

4.3 Lombok

Lombok 예시

```
import lombok.*;

@Data // @Getter + @Setter + @ToString + @EqualsAndHashCode + @RequiredArgsConstructor
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MemberDTO {

    private Long id;
    private String name;
    private String email;
    private int age;

    @Builder.Default
    private List<Employee> employees = new ArrayList<>();


}
```

수고하셨습니다.